Authenticated Diffie-Hellman Project Report

Prepared for the classroom project: Authenticated Diffie-Hellman Key Exchange with Library Prime Generation and Square-and-Multiply.

1. Goal and organization

This program simulates a signed Diffie-Hellman exchange between Alice and Bob inside one Python process. It generates public parameters, validates received parameters, calculates DH public values and secrets, and processes three signed messages. A separate Eve demonstration compares unauthenticated substitution with the authenticated rejection of the same substitution.

The implementation uses simple functions, dictionaries, loops, and separate modules. It implements modular exponentiation, factorization, generator tests, and DH calculations directly. PyCryptodome handles prime generation and primality validation. The cryptography package handles Ed25519 signatures. There is no custom signature implementation and no custom primality algorithm.

2. Square-and-multiply

`modular_exponentiation(base, exponent, modulus)` validates exact integer types, rejects Boolean inputs, negative exponents, and nonpositive moduli, and initializes `result = 1 % modulus` and `base = base % modulus`.

While the exponent is positive, an odd exponent causes the current base to be multiplied into the result. The base is then squared modulo the modulus, and the exponent is divided by two with integer division. Reducing after every multiplication avoids constructing the entire power. The function returns a number from zero through modulus minus one.

For positive exponent e, the loop has O(log e) iterations. This counts modular arithmetic operations; multiplying larger integers has its own additional cost. The implementation is not constant-time. The built-in three-argument pow is used only as an independent reference inside tests.

Worked example: 5^6 mod 23

Iteration	Exponent before	Base before	Result after odd check	Squared base mod 23	Next exponent
1	6	5	1	2	3
2	3	2	2	4	1
3	1	4	8	16	0

The result is 8. The boundary cases 5^0 mod 23 = 1 and 5^0 mod 1 = 0 are tested.

3. Library primes and generator selection

`generate_prime(bits=16)` accepts only integer sizes from 16 through 32 and rejects Boolean values. It calls getPrime with secrets.token_bytes and checks the returned bit length and oddness. Bob separately checks the received modulus type and size and calls the library's isPrime before factoring p - 1. The composite 32769 is rejected even though it is odd and has a permitted bit length.

The fixed p=23 example is enabled only by the explicit test-fixture path. Normal generated runs use exact 16-32-bit primes. A 16-bit request therefore produces a prime from 32768 through 65535, with no fixed prime hard-coded for that request.

`prime_factors(n)` divides out every factor of two, then tries odd divisors until divisor * divisor exceeds the remaining n. It records each factor once and records any remaining value greater than one. It returns an empty set for n=1. Thus, prime_factors(22) = {2, 11}, and prime_factors(65536) = {2}.

For a validated prime p, the multiplicative group has order p - 1. A candidate g has full order when g^((p - 1)/r) mod p is not 1 for every distinct prime factor r of p - 1. The search tries g from 2 through p - 2. The checking functions reject an incomplete supplied factor set rather than silently weakening the criterion.

For p=23, candidate 2 fails because 2^11 mod 23 = 1. Candidate 5 passes because 5^11 mod 23 = 22 and 5^2 mod 23 = 2. For p=65537, the distinct factor set is {2}, and candidate 3 passes because 3^32768 mod 65537 = 65536.

4. DH calculation and the fixed example

Each generated exchange uses fresh private exponents chosen with secrets.randbelow(p - 3) + 2. Public values are calculated using the custom square-and-multiply routine. A generated keypair is retried if the public value is outside 2 through p - 2; received public values are checked against the same range. Private exponents remain in local pending state and never enter a message.

The handout's mathematical fixture uses p=23, g=5, Alice's a=6, and Bob's b=15. The public values are A=8 and B=19. Alice calculates 19^6 mod 23 = 2, and Bob calculates 8^15 mod 23 = 2. These fixed values explain the math; they are not the parameters of the normal generated demonstration.

5. Trusted identities and message format

Alice and Bob each create a separate Ed25519 signing keypair. Trusted setup gives Alice Bob's genuine public signing key and gives Bob Alice's genuine public signing key before the exchange. The signing keys stay in memory across exchanges on the same participants. Incoming messages have no field for replacing trusted keys. Eve can alter simulated network messages but cannot access signing private keys or rewrite the trusted identity configuration.

Each message is a JSON envelope with exactly two fields: payload and signature. The signature is outside the signed payload and is represented by 128 hexadecimal characters. The payload bytes use sorted JSON keys, compact separators, and UTF-8. The fixed protocol label is DH-CLASSROOM-AUTH-v1.

Parsing rejects messages larger than 4096 bytes, duplicate JSON keys, unexpected fields, malformed hexadecimal nonces/signatures, nonstandard numeric constants, and wrong types. Integer fields have at most

32 bits before expensive arithmetic. Strings are never converted into integer fields. Identity labels must be exactly Alice and Bob. Each party generates a fresh 32-byte nonce encoded as 64 hex characters. Nonces are public identifiers for exchanges, not private keys.

6. Three authenticated messages and state

1. Alice signs an initiation containing protocol, message_type=init, initiator, responder, p, g, A, and nonce_A. Bob checks the schema and identities, verifies Alice's signature with the trusted key, independently validates p, factors p - 1, verifies g, and checks A.

2. Bob generates b, B, and nonce_B, then signs message_type=response with the complete transcript: protocol, initiator, responder, p, g, A, B, nonce_A, nonce_B. Alice checks every initiation field against her pending session, checks B, and verifies Bob's signature before calculating her secret.

3. Alice signs message_type=finish with the exact same complete transcript. Bob requires an active waiting-finish state, exact transcript equality, and a valid Alice signature. Only then does he calculate his secret and complete his side.

An error aborts the affected participant's session and clears pending state and usable shared-secret state. Ed25519 verification succeeds by returning normally; InvalidSignature is converted to an exchange rejection. Its return value is never treated as a Boolean success flag.

The coordinator compares the two resulting integers locally. It does not send either secret through a network message. Output distinguishes authenticated transcript status, local shared-secret equality, and explicit key confirmation. The third message authenticates Alice's acknowledgment; it is not explicit proof that Alice possesses the DH secret. Explicit key confirmation is not implemented.

7. Eve attacks and replay handling

In the unauthenticated fixture, Eve chooses e1=3 and e2=7, producing E1=10 and E2=17. She sends 10 to Alice as Bob's public value and 17 to Bob as Alice's public value. Alice computes 10^6 mod 23 = 6, matching Eve's 8^3 mod 23 = 6. Bob computes 17^15 mod 23 = 15, matching Eve's 19^7 mod 23 = 15. Alice and Bob have different secrets, and Eve knows both. The demonstration labels these local values as classroom debug output.

In the authenticated fixture, Eve changes B=19 to B=17 and keeps Bob's original signature. Alice rejects the altered response because the signature no longer matches the transcript. Her usable secret is cleared. The local demonstration coordinator also clears Bob's abandoned pending session; that cleanup is local, not an extra signed protocol message.

The test suite also checks altered A, p, g, identities and nonces, corrupted signatures at every stage, signatures made with Eve's key under Alice's label, stale responses, stale finishes, duplicate finishes, and malformed/oversized or out-of-order messages. A valid signature does not bypass independent parameter validation or exact pending-session checks.

There is no initiation replay cache. An old valid initiation can therefore cause Bob to issue a fresh response with a new nonce_B. The old finish cannot complete that new exchange because it contains the previous nonce_B. This distinction is covered by a specific automated test.

8. Actual tests and performance results

The recorded test execution passed 54 test methods, including many parameterized subtests. The suite is divided into fixed math tests, prime-generator integration, message validation, and complete exchanges. The independent arithmetic test compares 144 combinations against pow. Integration tests check every generated bit size from 16 through 32. Tests exercise the wrapper rather than the library's internal primality algorithm.

Nine additional generated authenticated exchanges were measured: three each at 16, 24, and 32 bits. All nine authenticated successfully and produced equal local secrets. The full rows are in results/performance.csv and results/performance.md.

Bits	Mean prime generation (s)	Mean factorization (s)	Mean generator search (s)
16	0.000247000	0.000002933	0.000017667
24	0.000441833	0.000029767	0.000038833
32	0.000660967	0.000225900	0.000252800

These measurements used time.perf_counter on Windows 11, AMD64 Family 25 Model 97 Stepping 2, with Python 3.12.14, cryptography 50.0.2, and PyCryptodome 4.2.0. They are actual observed durations, not promises about future executions. The detailed result file reports each stage's minimum, mean, and maximum. Random prime selection, the factors of p - 1, the first passing generator, and OS scheduling affect durations. Three runs per size are too few for broad performance conclusions.

Factorization is measured as its own stage. Generator search defensively recomputes the complete factor set, so that check is included in its measured duration. Bob's separate received-parameter validation is exercised in each full exchange but is not part of Alice's three parameter-generation timing columns.

9. Trust assumptions and limitations

The local process and preconfigured signing-key bindings are trusted. This project does not protect against a compromised process, stolen signing private keys, or an attacker who can rewrite the trusted setup. One active exchange per participant is supported; there are no sockets, concurrent session scheduler, or timeouts. Clearing Python references is logical state cleanup, not guaranteed memory erasure.

The intentionally small groups permit practical discrete-log recovery; digital signatures do not make them confidential. The custom arithmetic is not designed for side-channel resistance. Public-value range checking is the specified classroom check, not a complete production group-validation policy. No application message encryption or production key derivation is provided. ECDH, safe primes, sockets, and explicit key confirmation remain optional extensions.

The separate RESEARCH.md supplies the required 300-500-word comparison and authoritative references. CHECKLIST.md maps the completed implementation and saved evidence to every required deliverable.

Finite-field Diffie-Hellman compared with ECDH

The project's finite-field Diffie-Hellman operates on integers modulo a prime p. Alice publishes A = g^a mod p, Bob publishes B = g^b mod p, and both compute g^(ab) mod p. Square-and-multiply makes these exponentiations efficient. Elliptic-curve Diffie-Hellman instead uses scalar multiplication of curve points: Alice publishes aG, Bob publishes bG, and both obtain abG. The cryptography library provides ECDH as a key-agreement primitive and recommends passing its output through a key derivation function rather than using the raw result directly [1].

This assignment limits primes to 16-32 bits so that generated examples, debugging, and trial division finish quickly. These sizes are insecure: their tiny groups allow feasible recovery of private exponents by discrete-log attacks. The bit length of the modulus also does not directly equal the security strength. Signing the exchange can detect unauthorized changes, but it does not prevent an attacker from solving the small group's discrete logarithms.

Factoring p - 1 supplies the distinct prime factors needed to verify a primitive generator. The implemented trial-division method works well for the small classroom range, but scales poorly as p grows because candidate divisors can extend toward the square root of p - 1. This arbitrary-prime workflow is therefore unsuitable for choosing production-size parameters. ECDH uses established curve parameters and does not require this project's factorization of a newly generated p - 1.

Ephemeral ECDH, or ECDHE, generates a new key-agreement private value for each exchange. The library documentation explicitly distinguishes this from reusing a static private key [1]. With an appropriately authenticated protocol and proper handling of temporary secrets, fresh ephemeral keys support forward secrecy: later compromise of a long-term authentication key need not expose past session keys. RFC 9528's EDHOC is a concrete standardized example of authenticated ephemeral Diffie-Hellman with mutual authentication and forward secrecy [2].

ECDH still requires authentication. Without a trusted binding between an identity and its public credential, Eve can substitute public values and negotiate separate secrets with each participant. Ed25519 in this project signs the transcript; it is not the ECDH key-exchange operation. Production software combines established protocols, validated parameters, carefully implemented libraries, and contextual key derivation. As EDHOC illustrates, session binding and authentication are protocol responsibilities, not properties supplied by bare key agreement alone [2].

References

[1] Python Cryptographic Authority, "Elliptic curve cryptography," sections "Elliptic Curve Key Exchange algorithm" and the ephemeral ECDHE example. https://cryptography.io/en/stable/hazmat/primitives/asymmetric/ec/#elliptic-curve-key-exchange-algorithm

[2] G. Selander, J. Preuss Mattsson, and F. Palombini, RFC 9528, "Ephemeral Diffie-Hellman Over COSE (EDHOC)," March 2024, abstract, sections 2, 4.1.1.1, and 9.1. https://www.rfc-editor.org/rfc/rfc9528.html

Additional implementation references:

- PyCryptodome, Crypto.Util.number (getPrime and isPrime): https://www.pycryptodome.org/src/util/util#module-Crypto.Util.number

- Python, secrets module: https://docs.python.org/3/library/secrets.html

- Python Cryptographic Authority, Ed25519 signing: https://cryptography.io/en/stable/hazmat/primitives/asymmetric/ed25519/

Measured performance

These are actual local runs; timings vary.

Bits	Trial	Prime generation (s)	Factorization (s)	Generator search (s)	Authenticated	Secrets match
16	1	0.000246200	0.000003100	0.000018000	Yes	Yes
16	2	0.000220500	0.000003900	0.000017000	Yes	Yes
16	3	0.000274300	0.000001800	0.000018000	Yes	Yes
24	1	0.000447600	0.000004900	0.000016400	Yes	Yes
24	2	0.000449300	0.000073000	0.000078100	Yes	Yes
24	3	0.000428600	0.000011400	0.000022000	Yes	Yes
32	1	0.000541400	0.000276600	0.000297800	Yes	Yes
32	2	0.000544000	0.000203200	0.000226600	Yes	Yes
32	3	0.000897500	0.000197900	0.000234000	Yes	Yes

Timing variation

Ranges and means below describe these runs, not guaranteed runtimes.

- 16 bits, prime_generation_s: min 0.000220500, mean 0.000247000, max 0.000274300 seconds.
- 16 bits, factorization_s: min 0.000001800, mean 0.000002933, max 0.000003900 seconds.
- 16 bits, generator_search_s: min 0.000017000, mean 0.000017667, max 0.000018000 seconds.
- 24 bits, prime_generation_s: min 0.000428600, mean 0.000441833, max 0.000449300 seconds.
- 24 bits, factorization_s: min 0.000004900, mean 0.000029767, max 0.000073000 seconds.
- 24 bits, generator_search_s: min 0.000016400, mean 0.000038833, max 0.000078100 seconds.
- 32 bits, prime_generation_s: min 0.000541400, mean 0.000660967, max 0.000897500 seconds.
- 32 bits, factorization_s: min 0.000197900, mean 0.000225900, max 0.000276600 seconds.
- 32 bits, generator_search_s: min 0.000226600, mean 0.000252800, max 0.000297800 seconds.

Library prime generation varies with random candidates; factorization varies with p - 1; generator search varies with the first passing candidate. Timer resolution, scheduling, and cache effects also matter. This small sample is not a reliable scaling study.

Factorization and generator search are measured as separate stages. The generator function independently recomputes the complete factors for defensive checking, so its duration includes that verification. Bob's independent received-parameter validation occurs in each exchange but is not included in Alice's parameter-generation stage timings.

See environment.json for the exact Python, OS, processor, and dependency versions.